

RIM — Runtime Isolation Module

Parent Standard: Harness Architecture Standard

Parent Architecture: **Execution Layer — Enforcement Standard**

Category: Governance & Enforcement

Subcategory: Runtime Isolation

Type: Execution Containment Module

Version: 1.0

Status: Canonical · Open Module

Effective Date: 9 May 2026

Compatibility: OOF® Methodology OS™ · **Harness Architecture Standard** ·

Execution Layer — Enforcement Standard · RIS · INTEGROS® · OGL™ · ART · OIB™

Authority: OOF®

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

Runtime Isolation Module defines the structural conditions under which execution remains isolated from unrelated runtimes, sessions, processes, memory zones, contexts, or operational environments inside a **contained execution** architecture.

A system satisfies RIM only if:

- runtime separation is explicitly defined
- isolated execution zones are distinguishable
- cross-runtime interference is restricted
- containment between sessions, tasks, or processes remains governed
- execution does not silently cross from one runtime context into another without controlled boundary conditions

A system that allows execution to move across runtime contexts without governed isolation does not satisfy RIM.

Module Function

RIM defines the isolation layer of **contained execution**.

It ensures that execution does not remain merely bounded in theory, but also separated in practice from adjacent runtimes, unrelated sessions, unapproved contexts, or uncontrolled operational pathways.

The module applies wherever systems must preserve separation between:

- concurrent execution sessions
- distinct agents or workflows
- internal and external runtimes
- restricted and unrestricted operational contexts
- isolated task environments

- bounded memory and process spaces

Minimum Implementation Framework (MIF)

Step 1 — Define Runtime Domains

The organization must define which runtime domains exist inside the execution environment.

Minimum requirement:

- runtime domains are explicit
- isolated and non-isolated contexts are distinguishable
- undefined runtime zones are excluded from valid operation

Step 2 — Define Isolation Boundaries

The system must define how runtime contexts are separated.

Minimum requirement:

- isolation boundaries are explicit
- session, process, or task separation is identifiable
- silent boundary collapse is excluded from valid containment

Step 3 — Define Cross-Context Restrictions

The system must define when and how execution may interact across runtime boundaries.

Minimum requirement:

- cross-context interaction conditions are explicit
- unrestricted runtime crossover is excluded
- interaction between isolated zones does not occur by default

Step 4 — Define Containment Integrity Conditions

The system must define how isolation remains preserved during execution.

Minimum requirement:

- containment remains active during runtime
- boundary weakening, bleed, or shared-path leakage is detectable
- isolated execution does not silently merge into adjacent operational space

Step 5 — Preserve Runtime Traceability

The system must preserve visibility of where execution occurred and whether it crossed isolation boundaries.

Minimum requirement:

- runtime location and transition path are traceable
 - later review can reconstruct execution relative to isolated context
 - isolation does not depend on hidden assumptions
-

Step 6 — Restrict Invalid Runtime Operation

The system must not be treated as valid if execution proceeds across runtime boundaries without governed separation, controlled transfer, or traceable containment logic.

Minimum requirement:

- invalid isolation conditions are identifiable
 - cross-runtime spillover is blocked or invalidated
 - permission alone does not override runtime isolation
-

Use Case 1 — Multi-Agent Session Containment

Scenario

Multiple agents operate inside the same broader environment, but each must remain isolated from unrelated tasks, runtime state, and execution context unless controlled transfer is explicitly allowed.

Application

RIM is used to ensure:

- runtime domains are clearly defined
 - each execution session remains isolated
 - uncontrolled cross-session interaction is restricted
 - execution cannot silently move between agent contexts
-

Result

The organization gains a stronger basis for preventing **contained execution** from collapsing into shared uncontrolled runtime behavior.

Use Case 2 — Enterprise Runtime with Restricted Zones

Scenario

A system operates across internal execution areas with different levels of access, sensitivity, or operational role, and must ensure that execution inside one zone does not automatically affect another.

Application

RIM is used to ensure:

- runtime zones remain separated
 - boundary conditions are explicit
 - cross-zone execution is controlled
 - containment integrity remains visible during operation
-

Result

The organization gains a structurally governed runtime isolation layer instead of relying on assumed separation between execution contexts.

Canonical Closing Statement

If execution is not isolated, execution containment is not structurally real.

Related Documents

[→ Harness Architecture Standard](#)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>